

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Уфимский государственный нефтяной технический университет»

Кафедра «Вычислительная техника и инженерная кибернетика (ВТИК)»

Курсовой проект
По дисциплине «Искусственные нейронные сети»

Исполнитель:

студент гр. МПО09-25-01

В.А. Шамурзанов

Руководитель:

канд. техн. наук, доцент

В.М. Гиниятуллин

Уфа

2026

ВВЕДЕНИЕ

В задачах машинного обучения используются числовые и категориальные признаки. Способ цифрового представления категорий влияет на результат классификации, поэтому выбор кодировки представляет самостоятельную практическую задачу [1, 2].

В курсовом проекте исследуется возврат категорий в уже обученную сеть. Сначала модель обучается на числовых признаках, затем её рабочий ход воспроизводится в Excel. Вводится дополнительный вход с единичными весами и подбираются значения категорий; исходные веса сети сохраняются.

Использован файл `Daily_Water_Intake.csv`, вариант 17. Целевая переменная — Hydration Level (Poor, Good); возвращаемые категории — пол, активность и погода. Методика основана на задании «Кодировка категорий» и предоставленном Excel-примере [5–7].

Цель курсового проекта — исследовать возможность уменьшения числа ошибок бинарной классификации за счёт цифровой кодировки категориальных признаков при фиксированных весах нейронной сети.

Для достижения цели в работе необходимо:

1. Подготовить данные и обучить ReLU и Tanh без категорий.
2. Перенести рабочий ход в Excel и сверить предсказания.
3. Ввести дополнительный вход и подобрать коды категорий.
4. Исследовать отдельное и совместное влияние категорий.
5. Оценить качество на независимой тестовой выборке.

Подбор коэффициентов автоматизирован на Python. Значения перенесены в редактируемые ячейки Excel для ручного исследования. Отчёт описывает фактически выполненный программный поиск.

1. ПОСТАНОВКА ЗАДАЧИ

Решается задача бинарной классификации уровня гидратации по данным варианта 17. Нулевая метка соответствует Poor, единичная — Good. Для базовой сети используются три числовых признака: Age, Weight (kg), Daily Water Intake (liters). Три категориальных признака полностью исключаются из обучения и сохраняются отдельно для последующего эксперимента.

Обученная модель имеет структуру $3 \rightarrow 10 \rightarrow 1$. Для включения категорий без повторного обучения формируется величина q — сумма кодов пола, активности и погоды. Эта величина поступает на каждый из десяти скрытых нейронов с фиксированным весом 1. Число скрытых и выходных нейронов не изменяется; число входов расширяется с трёх до четырёх.

Основной критерий подбора — число ошибок на проверочной выборке. Accuracy, Precision, Recall и F1 используются для сопоставления моделей. Порог вероятности равен 0,5. Условие $\text{logit} \geq 0$ эквивалентно отнесению объекта к классу Good.

Качество оценивается отдельно на обучающей, проверочной и тестовой частях. Проверочная выборка используется для выбора эпохи и кодов; тестовая — для итоговой оценки. Такой порядок отделяет подбор параметров от проверки результата [4].

2. ОПИСАНИЕ ИСХОДНЫХ ДАННЫХ

2.1. Структура датасета

Исходный CSV-файл содержит 30000 строк и 7 столбцов. Строк с пропущенными значениями обнаружено 0. Удалено 338 полных дубликатов; после очистки осталось 29662 наблюдения. Номера исходных строк сохранены в поле `source_csv_row`, что позволяет проверить местоположение каждой ошибки.

Целевая переменная закодирована следующим образом: `Poor` = 0, `Good` = 1. После очистки класс `Poor` содержит 5 932 объекта, класс `Good` — 23 730 объектов. Большинство наблюдений относится к классу `Good`; простое предсказание этого класса даёт около 80% правильных ответов.

2.2. Числовые и категориальные признаки

Таблица 1 — Признаки набора данных

Признак	Содержание	Использование
Age	Возраст	Обучение базовой сети
Weight (kg)	Масса тела	Обучение базовой сети
Daily Water Intake (liters)	Потребление воды	Обучение базовой сети
Gender	Female, Male	Подбор 2 кодов
Physical Activity Level	Low, Moderate, High	Подбор 3 кодов
Weather	Cold, Normal, Hot	Подбор 3 кодов
Hydration Level	Poor, Good	Целевая переменная

Масса тела сохранена как числовой признак. Ограничение на четыре выбранных признака из предыдущей лабораторной работы здесь не применяется: задание №4 требует исключить категории, а не уменьшить число числовых входов.

2.3. Разделение и нормализация

Таблица 2 — Состав выборок

Выборка	Объектов	Poor	Good
Обучающая (60%)	17796	3559	14237
Проверочная (20%)	5933	1186	4747
Тестовая (20%)	5933	1187	4746

Разбиение стратифицировано, `random_state = 42`. Вначале выделяется 20% тестовых данных, затем четверть оставшейся части отводится под `validation`. Средние значения и стандартные отклонения рассчитываются только на обучающей части с помощью `StandardScaler` [4].

Таблица 3 — Параметры стандартизации

Признак	Среднее	Стандартное отклонение
Age	43,3015	15,0079
Weight (kg)	77,0474	18,6463
Daily Water Intake (liters)	2,8632	0,8229

Вычисление нормализованного входа: $x = (v - \mu) / \sigma$. Перед обучением входы приведены к `float32`; эти же значения экспортированы в Excel. Категориальные коды добавляются к преактивациям после нормализации числовых данных и отдельно не стандартизируются.

3. БАЗОВАЯ НЕЙРОННАЯ СЕТЬ

3.1. Архитектура сети

Построены две модели с одинаковой структурой: три входа, скрытый слой из десяти нейронов и один выходной нейрон. Модели различаются функцией активации скрытого слоя — ReLU либо Tanh. Выходной слой формирует logit; sigmoid применяется при вычислении вероятности [3].

Прямой проход записывается в виде $z = Wx + b$; $h = \varphi(z)$; $l = v^T h + b_0$; $p = 1 / (1 + \exp(-l))$. Для ReLU используется $\varphi(z) = \max(0, z)$, для Tanh — гиперболический тангенс.

3.2. Гиперпараметры обучения

Таблица 4 — Параметры обучения

Параметр	Значение
Функция потерь	BCEWithLogitsLoss
Оптимизатор	Adam
Learning rate	0,001
Batch size	256
Максимальное число эпох	150
Patience	20 эпох
Критерий сохранения	Минимальный validation loss
Seed	42

Обе модели обучались независимо. Категориальные столбцы не передавались в обучающий набор. Веса сохранялись при уменьшении проверочной функции потерь более чем на 10^{-6} ; тестовые показатели не использовались для выбора эпохи.

Для обеих моделей лучшим оказалось состояние на 150-й эпохе. Обучение завершилось по ограничению числа эпох, а не по срабатыванию ранней остановки. Это означает, что полученные модели являются результатом заданного вычислительного эксперимента, но не доказанным оптимумом обучения.

3.3. Код модели (Python/PyTorch)

```
class WaterNet(nn.Module):
    def __init__(self, activation):
        super().__init__()
        self.hidden = nn.Linear(3, 10)
        self.activation = nn.ReLU() if activation == "ReLU" else nn.Tanh()
        self.output = nn.Linear(10, 1)
    def forward(self, x):
        return self.output(self.activation(self.hidden(x))).squeeze(1)
```

3.4. Результаты обучения

Таблица 5 — Базовые модели на тестовой выборке

Метрика	ReLU	Tanh
accuracy	99,9494%	99,8820%
precision	99,9368%	99,8527%
recall	100,0000%	100,0000%
f1	99,9684%	99,9263%
Ошибок	3	7

ReLU допустила 3 ошибки, Tanh — 7 ошибок. Все ошибки исходных моделей на тесте относятся к ложноположительным: объекты Poor классифицированы как Good. Базовые сети уже обеспечивают высокую точность, поэтому ожидаемый резерв улучшения невелик.

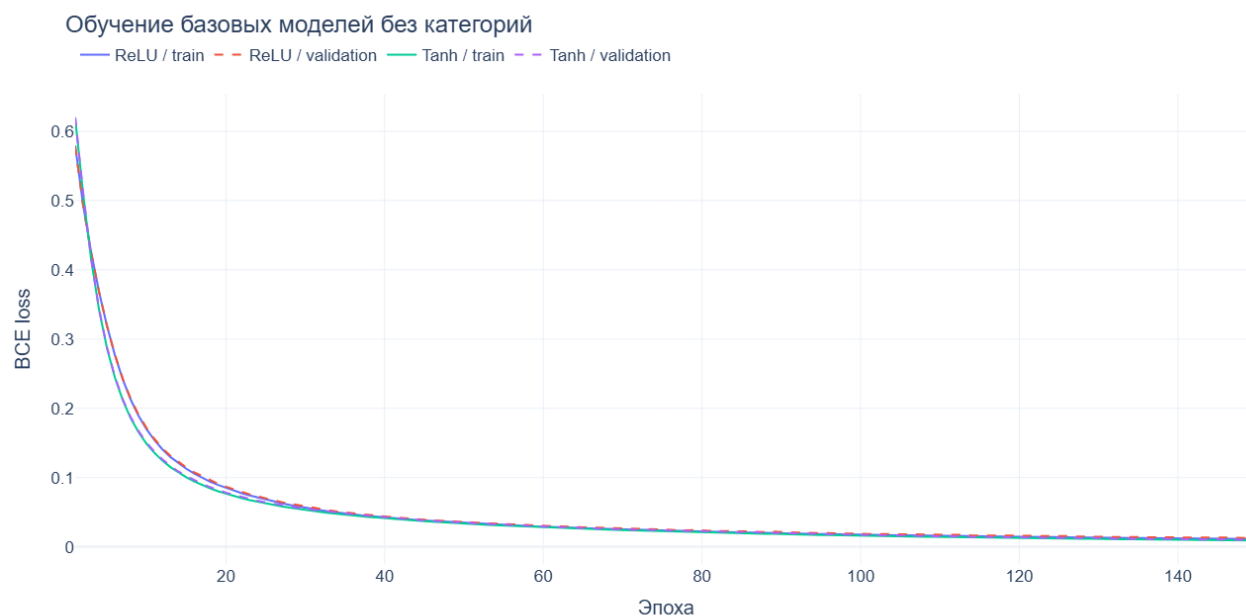


Рисунок 1 — Изменение функции потерь при обучении ReLU и Tanh

4. РЕАЛИЗАЦИЯ В EXCEL

4.1. Структура файлов

Создана одна рабочая книга water_variant17_lab4_work.xlsx. В ней объединены исходные данные, веса обеих сетей, редактируемые коды и четыре варианта прямого прохода. Расчётные листы содержат всю тестовую выборку — 5 933 объекта, без усечения до первых строк.

Таблица 6 — Основные листы рабочей книги

Лист	Содержание
Summary	Сводные формулы ошибок, ассигнату и проверки
Raw_data	Все 29 662 очищенные строки и принадлежность к выборке
Input	Тестовые строки, категории и нормализованные числовые входы
Normalization	Средние и масштабы обучающей части
Codes	Восемь значений категорий для каждой модели
Weights_ReLU / Weights_Tanh	Весовые матрицы, смещения и единичная строка
ReLU_baseline / Tanh_baseline	Прямой проход с нулевым дополнительным входом
ReLU_tuned / Tanh_tuned	Прямой проход с выбранными кодами
Interactions	Результаты раздельного и совместного подбора

4.2. Воспроизведение рабочего хода

Числовой вход умножается на веса первого слоя с добавлением смещений. Затем применяются $\text{MAX}(0, z)$ либо $\text{TANH}(z)$, SUMPRODUCT для выходного слоя, sigmoid и пороговое правило. Параметры первого слоя экспортированы без округления, чтобы сохранить результат вычисления.

В транспонированную матрицу первого слоя добавлена строка из десяти единиц. Во входной вектор добавлен столбец q , изначально равный нулю. При таком расширении вклад новых связей равен нулю, поэтому исходные logit , метки и ошибки сохраняются. После подстановки кодов формула принимает вид $z_j = b_j + \sum_i w_{ji} x_i + q$.

```
q = code(Gender) + code(Activity) + code(Weather)
z_j = bias_j + x_age*w_age_j + x_weight*w_weight_j + x_water*w_water_j + q
h_j = MAX(0, z_j) или TANH(z_j)
```

```
logit = SUMPRODUCT(h_1:h_10, output_weights) + output_bias
prediction = IF(logit >= 0, 1, 0)
error = IF(prediction <> target, 1, 0)
```

4.3. Проверка правильности расчёта

Рабочая книга открыта и полностью пересчитана Microsoft Excel. После сохранения числовые значения формул прочитаны обратно и сопоставлены с предсказаниями Python для каждой исходной строки CSV. Проверялись не только суммы ошибок, но и их номера.

Таблица 7 — Сверка Excel и Python

Вариант	Проверено строк	Расхождений меток	Ошибок
ReLU_baseline	5933	0	3
ReLU_tuned	5933	0	4
Tanh_baseline	5933	0	7
Tanh_tuned	5933	0	0

Количество и местоположение ошибок совпали для всех четырёх вариантов. Максимальное абсолютное различие logit между сохранёнными CSV-значениями Python и Excel составляет менее 10^{-12} . Хеши весов до и после подбора совпадают. Результаты проверки сохранены в excel_verification.json.

Для ручного исследования необходимо изменить жёлтые ячейки листа Codes и пересчитать книгу. Формулы tuned обновятся, а столбцы Python_prediction останутся эталоном первоначально найденного решения. Поэтому после ручных изменений отличия от этого эталона ожидаемы.

5. ПОДБОР КОЭФФИЦИЕНТОВ ДЛЯ ТРЁХ ПРИЗНАКОВ

5.1. Методика

Для пола задаются два коэффициента, для активности и погоды — по три, всего восемь значений. По строке данных выбирается одно значение каждого признака; их сумма образует дополнительный вход q . Этот способ реализует единичную строку весов из методического примера [6, 7].

Подбор выполнен дифференциальной эволюцией [8] в диапазоне от -2 до 2 . Использованы $\text{seed} = 42$, до 45 поколений и множитель размера популяции 6. Затем проведено покоординатное уточнение с шагами 0,1; 0,02; 0,005. Экспортируемые коэффициенты округлены до трёх десятичных знаков; качество пересчитано именно с этими значениями.

Целевая функция равна числу ошибок validation с малой добавкой $10^{-6}\Sigma c^2$. Эта добавка лишь разрешает равенство числа ошибок в пользу меньших коэффициентов. Нулевая кодировка включена в сравнение, поэтому ухудшающая validation настройка не принимается.

Исследованы семь наборов допустимых изменений: каждый признак отдельно, каждая пара и все три одновременно. Для каждой модели выбиралась первая настройка с минимальным числом проверочных ошибок в порядке этого перебора. Это локальный вычислительный поиск; глобальная оптимальность найденных коэффициентов не утверждается.

5.2. Итоговые коэффициенты

Таблица 8 — Выбранные цифровые коды

Признак	Категория	ReLU	Tanh
Gender	Female	-0,007	-0,011
Gender	Male	-0,007	-0,015
Physical Activity Level	Low	0,000	0,000
Physical Activity Level	Moderate	0,000	0,000
Physical Activity Level	High	0,000	0,000
Weather	Cold	-0,012	0,000

Признак	Категория	ReLU	Tanh
Weather	Normal	-0,015	0,000
Weather	Hot	0,000	0,000

Для Tanh выбрана настройка только признака Gender: Female = $-0,011$, Male = $-0,015$. Коды активности и погоды равны нулю. Она уменьшила число ошибок validation с 11 до 0 и число тестовых ошибок с 7 до 0.

Для ReLU выбрана совместная настройка пола и погоды. Оба значения пола получили код $-0,007$, то есть эта составляющая является общей добавкой и не различает категории. Погода задаёт коды Cold = $-0,012$, Normal = $-0,015$, Hot = 0. Активность не используется. Ошибки validation уменьшились с 6 до 0, но число тестовых ошибок выросло с 3 до 4.

Одинаковые или близкие коды нельзя трактовать как доказательство самостоятельной информативности категорий. В частности, эффект Gender у Tanh может преимущественно отражать небольшую общую корректировку преактиваций. В данной схеме существенна сумма трёх кодов; отдельные коэффициенты не имеют единственной интерпретации.

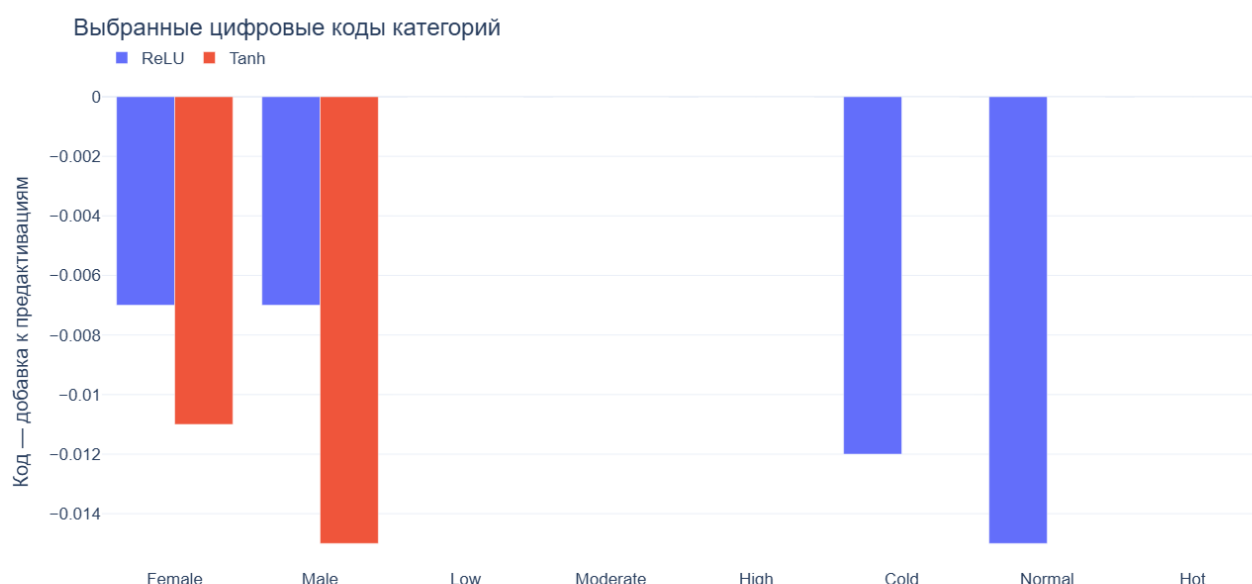


Рисунок 2 — Значения выбранных кодов для ReLU и Tanh

5.3. Взаимовлияние категориальных столбцов

Таблица 9 — Раздельный и совместный подбор

Изменяемые признаки	ReLU val / test	Tanh val / test
Пол	2 / 1	0 / 0
Активность	1 / 1	0 / 1
Погода	1 / 1	0 / 0
Пол + Активность	1 / 9	0 / 0
Пол + Погода	0 / 4	0 / 0
Активность + Погода	2 / 9	0 / 0
Пол + Активность + Погода	2 / 1	0 / 0

В таблице приведены ошибки на проверочной и тестовой выборках. Каждая строка соответствует самостоятельному подбору при фиксированных остальных кодах, равных нулю. Влияние пары не обязано совпадать с суммой отдельных эффектов: складывающиеся коды проходят через нелинейную функцию активации и порог классификации.

Результаты строк таблицы используются для описания эксперимента. Итоговая кодировка выбиралась по validation; дополнительные тестовые результаты не использовались для повторного подбора коэффициентов.



Рисунок 3 — Число ошибок validation при разных сочетаниях категориальных признаков

6. РЕЗУЛЬТАТЫ

6.1. Метрики качества классификации

Accuracy — доля правильных ответов. Precision — доля истинных объектов Good среди предсказанных Good. Recall — доля найденных объектов Good среди всех объектов этого класса. F1 — гармоническое среднее Precision и Recall. Строки матрицы ошибок соответствуют истинным классам, столбцы — предсказанным, порядок классов: Poor, Good.

6.2. Результаты на тестовой выборке

Таблица 10 — Метрики на 5 933 тестовых объектах

Метрика	ReLU до	ReLU после	Tanh до	Tanh после
accuracy	99,9494%	99,9326%	99,8820%	100,0000%
precision	99,9368%	99,9789%	99,8527%	100,0000%
recall	100,0000%	99,9368%	100,0000%	100,0000%
f1	99,9684%	99,9579%	99,9263%	100,0000%
Ошибок	3	4	7	0

ReLU, до подбора: TN = 1184, FP = 3, FN = 0, TP = 4746. Accuracy = 99,9494%; ошибок — 3.

ReLU, после подбора: TN = 1186, FP = 1, FN = 3, TP = 4743. Accuracy = 99,9326%; ошибок — 4.

Tanh, до подбора: TN = 1180, FP = 7, FN = 0, TP = 4746. Accuracy = 99,8820%; ошибок — 7.

Tanh, после подбора: TN = 1187, FP = 0, FN = 0, TP = 4746. Accuracy = 100,0000%; ошибок — 0.

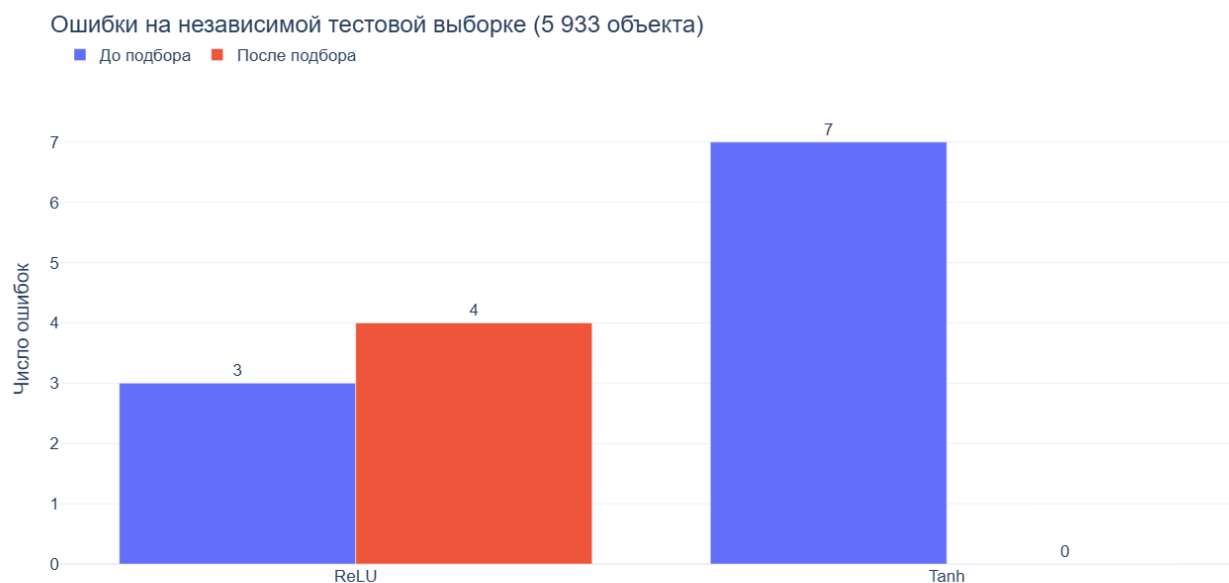


Рисунок 4 — Сравнение числа ошибок до и после подбора кодов

6.3. Результаты на полном наборе данных

Таблица 11 — Описательная оценка на всех очищенных данных

Модель	До: ошибок / accuracy	После: ошибок / accuracy
ReLU	17 / 99,9427%	9 / 99,9697%
Tanh	32 / 99,8921%	0 / 100,0000%

Полная выборка содержит обучающие и проверочные объекты, поэтому её показатели приведены как описательная характеристика. Независимой итоговой оценкой остаётся результат на выделенном тесте.

6.4. Сравнительный анализ и ограничения

У Tanh подбор устранил семь тестовых ошибок и повысил accuracy на 0,1180 процентного пункта. У ReLU результат на тесте ухудшился на одну ошибку, несмотря на отсутствие ошибок validation. Следовательно, уменьшение проверочной ошибки само по себе не гарантирует улучшения на новых объектах.

Обе настроенные модели получили ноль ошибок validation. Правило выбора по этому показателю с сохранением первой модели при равенстве выбрало бы ReLU. Tanh имеет лучший наблюдаемый результат на тесте, но это не следует

выдавать за результат выбора архитектуры на тестовых данных. В работе приведены обе заранее заданные архитектуры.

Базовые сети уже разделяют данные почти безошибочно. Поэтому влияние найденных кодов невелико и относится к нескольким пограничным объектам. Нулевое число ошибок Tanh в данном разбиении не означает гарантированную точность 100% на любых данных. Для вывода об устойчивости эффекта нужны повторные разбиения и независимые данные; в этом эксперименте они не исследовались.

Высокая точность характеризует воспроизведение меток предоставленного CSV. Работа не проверяет способ получения исходных меток и не устанавливает применимость модели для медицинской диагностики. Поиск кодов не доказывает причинного влияния пола, активности или погоды на гидратацию.

7. ЗАКЛЮЧЕНИЕ

В ходе курсового проекта разработана и исследована схема включения категориальных признаков в обученную нейронную сеть при фиксированных весах на данных Daily Water Intake, вариант 17.

Выполнены следующие задачи:

1. Подготовлены данные: удалены 338 дубликатов, сохранены 29 662 наблюдения, сформированы обучающая, проверочная и тестовая выборки.
2. Обучены сети $3 \rightarrow 10 \rightarrow 1$ с активациями ReLU и Tanh на трёх числовых признаках; категориальные признаки из обучения исключены.
3. Реализован прямой проход в Excel. Для всех 5 933 тестовых объектов совпали предсказания и номера ошибок Python и электронной таблицы.
4. Введён дополнительный вход с единичными весами. Подобраны коды пола, активности и погоды при неизменных исходных параметрах сетей.
5. Проверено раздельное и совместное влияние трёх категориальных столбцов; построены графики обучения, коэффициентов и ошибок.
6. Для Tanh число тестовых ошибок уменьшено с 7 до 0; Accuracy, Precision, Recall и F1 на данном тесте составили 100%. Для ReLU ошибки изменились с 3 до 4.
7. Сохранены модели, рабочая книга Excel, таблицы предсказаний и метрик, результаты проверки и иллюстрации.

В исследованном разбиении возможность улучшения классификации без переобучения исходных весов показана для Tanh. Результат ReLU демонстрирует ограничение метода: настройка по validation может ухудшить тестовое качество. Поэтому эффект кодировки необходимо проверять на данных, не использованных при её подборе.

Для дальнейшей проверки эффекта необходимы повторные разбиения и сравнение с постоянной добавкой к преактивациям.

СПИСОК ЛИТЕРАТУРЫ

1. Goodfellow I., Bengio Y., Courville A. Deep Learning. MIT Press, 2016.
<https://www.deeplearningbook.org/>
2. Chollet F. Deep Learning with Python. Manning Publications, 2018.
3. Документация PyTorch: <https://pytorch.org/docs/>
4. Документация scikit-learn: <https://scikit-learn.org/> ; Common pitfalls: https://scikit-learn.org/stable/common_pitfalls.html
5. Daily Water Intake & Hydration Patterns Dataset. Предоставленный файл Daily_Water_Intake.csv, вариант 17. Исходные данные расчёта.
6. Кодировка категорий. Методические указания к лабораторной работе. Файл «кодировкаКатегорий_26_02_15.docx», предоставленный с заданием.
7. Пример реализации рабочего хода нейронной сети в электронных таблицах. Файл «lab_4_work_26_02_17.xlsx», предоставленный с заданием.
8. Документация SciPy. `scipy.optimize.differential_evolution`: https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.differential_evolution.html

ПРИЛОЖЕНИЯ

Приложение А. Основной код расчёта

Файл lab4_recalculation.py. Модуль формирования Excel — lab4_workbook.py;
сборка отчёта — build_lab4_report.py. Полные версии сохранены в каталоге проекта.

```

"""ЛР4: числовая сеть и возврат категорий через вход с единичными весами."""
from __future__ import annotations
import argparse
import hashlib
import itertools
import json
import copy
import deepcopy
from pathlib import Path
import numpy as np
import pandas as pd
import torch
from scipy.optimize import differential_evolution
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from torch import nn

SEED = 42
NUMERIC = ['Age', 'Weight (kg)', 'Daily Water Intake (liters)']
CATEGORIES = {'Gender': ['Female', 'Male'], 'Physical Activity Level': ['Low', 'Moderate', 'High'], 'Weather': ['Cold', 'Normal', 'Hot']}
KEYS = [(field, value) for field, values in CATEGORIES.items() for value in values]
MAX_EPOCHS, PATIENCE = 150, 20

class WaterNet(nn.Module):
    def __init__(self, activation):
        super().__init__()
        self.hidden = nn.Linear(3, 10)
        self.activation = nn.ReLU() if activation == 'ReLU' else nn.Tanh()
        self.output = nn.Linear(10, 1)
    def forward(self, x):
        return self.output(self.activation(self.hidden(x))).squeeze(1)

def assess(y, logits):
    pred = (logits >= 0).astype(int)
    return {'n': len(y), 'errors': int(np.sum(pred != y)),
            'accuracy': float(accuracy_score(y, pred)),
            'precision': float(precision_score(y, pred, zero_division=0)),
            'recall': float(recall_score(y, pred, zero_division=0)),
            'f1': float(f1_score(y, pred, zero_division=0)),
            'loss': float(np.mean(np.logaddexp(0, logits) - y * logits)),
            'confusion_matrix': confusion_matrix(y, pred, labels=[0, 1]).tolist()}

def forward_numpy(x, w, activation, shift=None):
    z = x @ w['hidden_weight'].T + w['hidden_bias']
    if shift is not None: z = z + shift[:, None]
    h = np.maximum(z, 0) if activation == 'ReLU' else np.tanh(z)
    return z, h, h @ w['output_weight'] + w['output_bias']

def train(x, y, splits, activation):
    torch.manual_seed(SEED)
    model = WaterNet(activation)
    opt = torch.optim.Adam(model.parameters(), lr=0.001)
    loss_fn = nn.BCEWithLogitsLoss()
    tx, ty = torch.tensor(x, dtype=torch.float32), torch.tensor(y, dtype=torch.float32)
    loader = torch.utils.data.DataLoader(torch.utils.data.TensorDataset(tx[splits['train']], ty[splits['train']])),
        batch_size=256, shuffle=True, generator=torch.Generator().manual_seed(SEED))
    best_loss, bad, best_epoch, history = float('inf'), 0, 0, []

```

```

for epoch in range(1, MAX_EPOCHS + 1):
    model.train()
    for bx, by in loader:
        opt.zero_grad()
        loss_fn(model(bx), by).backward()
        opt.step()
    model.eval()
    with torch.no_grad():
        row = {'activation': activation, 'epoch': epoch}
        for name in ('train', 'validation'):
            idx = splits[name]
            scores = assess(y[idx], model(tx[idx]).numpy())
            row[name+'_loss'], row[name+'_accuracy'] = scores['loss'], scores['accuracy']
        history.append(row)
    if row['validation_loss'] < best_loss - 1e-6:
        best_loss, best_epoch, bad = row['validation_loss'], epoch, 0
        best_state = deepcopy(model.state_dict())
    else: bad += 1
    if bad >= PATIENCE: break
model.load_state_dict(best_state)
w = {'hidden_weight': model.hidden.weight.detach().numpy().astype(float),
     'hidden_bias': model.hidden.bias.detach().numpy().astype(float),
     'output_weight': model.output.weight.detach().numpy().ravel().astype(float),
     'output_bias': float(model.output.bias.item())}
return model, w, history, best_epoch

def optimize_codes(z, design, y, weights, activation, allowed):
    """Коды подбираются только на validation, веса и нормализация фиксированы."""
    columns = [i for i, (field, _) in enumerate(KEYS) if field in allowed]
    evaluations = 0
    def objective(values):
        nonlocal evaluations
        evaluations += 1
        shift = np.sum(design[:, columns] * values, axis=1)
        shifted = z + shift[:, None]
        h = np.maximum(shifted, 0) if activation == 'ReLU' else np.tanh(shifted)
        pred = np.sum(h * weights['output_weight'], axis=1) + weights['output_bias']
        return float(np.sum(pred != y) + 1e-6 * np.sum(values ** 2))
    result = differential_evolution(objective, [(-2, 2)] * len(columns), seed=SEED,
                                   maxiter=45, popsize=6, polish=False, tol=0, atol=0, x0=np.zeros(len(columns)))
    values = np.round(result.x, 3)
    if objective(np.zeros(len(columns))) <= objective(values): values[:] = 0
    for step in (0.1, 0.02, 0.005):
        for _ in range(3):
            old = values.copy()
            for j in range(len(columns)):
                trials = []
                for delta in range(-4, 5):
                    candidate = values.copy()
                    candidate[j] = np.round(np.clip(values[j] + delta * step, -2, 2), 3)
                    trials.append((objective(candidate), candidate))
                values = min(trials, key=lambda item: item[0])[1]
            if np.array_equal(old, values): break
    full = np.zeros(len(KEYS))
    full[columns] = values
    return full, evaluations

def run(data_path, output):
    output.mkdir(parents=True, exist_ok=True)
    torch.set_num_threads(2)
    np.random.seed(SEED)
    raw = pd.read_csv(data_path)
    frame = raw[NUMERIC + list(CATEGORIES) + ['Hydration Level']].copy()
    for field in list(CATEGORIES) + ['Hydration Level']: frame[field] = frame[field].str.strip()
    missing = int(frame.isna().any(axis=1).sum())
    frame = frame.dropna()
    duplicates = int(frame.duplicated().sum())
    frame = frame.loc[~frame.duplicated()].copy()
    frame.insert(0, 'source_csv_row', frame.index + 2)
    frame.reset_index(drop=True, inplace=True)
    for field, values in CATEGORIES.items():
        if not frame[field].isin(values).all(): raise ValueError(f'Unknown category in {field}')
    target = frame['Hydration Level'].map({'Poor': 0, 'Good': 1})

```

```

if target.isna().any(): raise ValueError('Unknown target')
y = target.to_numpy(dtype=int)
trainval, test = train_test_split(np.arange(len(frame)), test_size=0.2, stratify=y, random_state=SEED)
train_idx, val = train_test_split(trainval, test_size=0.25, stratify=y[trainval], random_state=SEED)
splits = {'train': train_idx, 'validation': val, 'test': test}
scaler = StandardScaler().fit(frame.loc[train_idx, NUMERIC])
x = scaler.transform(frame[NUMERIC]).astype(np.float32).astype(float)
design = np.column_stack([(frame[f] == v).to_numpy(dtype=float) for f,v in KEYS])
summary = {'variant':17, 'seed':SEED, 'raw_rows':len(raw), 'missing_rows':missing, 'duplicates_removed':duplicates,
           'rows':len(frame), 'numeric_features':NUMERIC, 'categories':CATEGORIES, 'category_keys':KEYS,
           'class_counts':np.bincount(y).tolist(),
           'splits':{k:{'n':len(v), 'class_counts':np.bincount(y[v]).tolist()} for k,v in splits.items()},
           'normalization_mean':scaler.mean_.tolist(), 'normalization_std':scaler.scale_.tolist(),
           'data_sha256':hashlib.sha256(data_path.read_bytes()).hexdigest(), 'models':{}}
predictions = pd.DataFrame({'source_csv_row':frame.source_csv_row, 'target':y, 'split':''})
for name,idx in splits.items(): predictions.loc[idx, 'split'] = name
all_interactions, all_history = [], []
for activation in ('ReLU', 'Tanh'):
    print(f'Training {activation} on numeric features only...', flush=True)
    model, w, history, best_epoch = train(x, y, splits, activation)
    z, h, baseline_logits = forward_numpy(x, w, activation)
    with torch.no_grad(): pytorch_logits = model(torch.tensor(x, dtype=torch.float32)).numpy()
    mismatch = int(np.sum((pytorch_logits>=0) != (baseline_logits>=0)))
    assert mismatch == 0, 'PyTorch / formula predictions differ'
    assert np.array_equal(forward_numpy(x, w, activation, np.zeros(len(x)))[2], baseline_logits)
    weight_hash = hashlib.sha256(b''.join(p.detach().numpy().tobytes() for p in model.parameters())).hexdigest()
    best_codes, best_errors = np.zeros(len(KEYS)), assess(y[val], baseline_logits[val])['errors']
    rows = []
    for count in (1,2,3):
        for fields in itertools.combinations(CATEGORIES, count):
            print(f' {activation}: tuning {" + ".join(fields)} on validation...', flush=True)
            codes, evaluations = optimize_codes(z[val], design[val], y[val], w, activation, fields)
            logits = forward_numpy(x, w, activation, design @ codes)[2]
            scores = {name:assess(y[idx], logits[idx]) for name,idx in splits.items()}
            rows.append({'activation':activation, 'fields':list(fields), 'codes':codes.tolist(),
                        'evaluations':evaluations, 'validation':scores['validation'], 'test':scores['test']})
            if scores['validation']['errors'] < best_errors:
                best_codes, best_errors = codes, scores['validation']['errors']
            tuned_logits = forward_numpy(x, w, activation, design @ best_codes)[2]
            assert weight_hash == hashlib.sha256(b''.join(p.detach().numpy().tobytes() for p in
model.parameters())).hexdigest()
            metrics = {'best_epoch':best_epoch, 'epochs_run':len(history), 'codes':best_codes.tolist(),
                      'baseline':{name:assess(y[idx], baseline_logits[idx]) for name,idx in splits.items()},
                      'tuned':{name:assess(y[idx], tuned_logits[idx]) for name,idx in splits.items()},
                      'all_baseline':assess(y, baseline_logits), 'all_tuned':assess(y, tuned_logits),
                      'verification':{'pytorch_numpy_prediction_mismatches':mismatch,
                                     'max_logit_difference':float(np.max(np.abs(pytorch_logits-baseline_logits))),
                                     'zero_input_preserves_logits':True, 'weights_sha256_before_after':weight_hash},
                      'weights':{k:v.tolist() if isinstance(v, np.ndarray) else v for k,v in w.items()}}
            summary['models'][activation] = metrics
            for name, logits in [(('baseline', baseline_logits), ('tuned', tuned_logits))]:
                predictions[f'{activation}_{name}_logit'] = logits
                predictions[f'{activation}_{name}_pred'] = (logits>=0).astype(int)
                predictions[f'{activation}_{name}_error'] = (logits>=0) != y
            all_interactions.extend(rows)
            all_history.extend(history)
            torch.save({'model_state_dict':model.state_dict(), 'numeric_features':NUMERIC,
                      'category_keys':KEYS, 'codes':best_codes.tolist(), 'scaler_mean':scaler.mean_.tolist(),
                      'scaler_scale':scaler.scale_.tolist(), 'activation':activation, 'hidden_neurons':10}, output/f'water_{activation}_lab4.
pth')
            print(f" {activation} test: {metrics['baseline']['test']['errors']} -> {metrics['tuned']['test']['errors']}
errors", flush=True)
            summary['interactions'] = all_interactions
            summary['selected_model'] = min(summary['models'], key=lambda
a:summary['models'][a]['tuned']['validation']['errors'])
            (output/'metrics.json').write_text(json.dumps(summary, ensure_ascii=False, indent=2), encoding='utf-8')
            predictions.to_csv(output/'predictions.csv', index=False)
            pd.DataFrame(all_history).to_csv(output/'training_metrics.csv', index=False)
            pd.DataFrame([{'activation':r['activation'], 'fields':r['fields'],
                          'validation_errors':r['validation']['errors'], 'test_errors':r['test']['errors'],
                          'codes':json.dumps(r['codes'])} for r in
all_interactions]).to_csv(output/'category_interactions.csv', index=False)
            from lab4_workbook import save_workbook
            save_workbook(output/'water_variant17_lab4_work.xlsx', frame, x, y, splits, summary, predictions)

```

```
print('Finished: '+str(output.resolve()),flush=True)

if __name__ == '__main__':
    parser=argparse.ArgumentParser()
    parser.add_argument('--data',type=Path,default=Path('Daily_Water_Intake.csv'))
    parser.add_argument('--output-dir',type=Path,default=Path('lab4_reworked'))
    args=parser.parse_args()
    run(args.data,args.output_dir)
```

Приложение Б. Местоположение тестовых ошибок

ReLU_baseline: 27305, 13769, 16388.

ReLU_tuned: 8436, 27305, 17353, 16778.

Tanh_baseline: 20184, 27305, 13769, 11283, 16388, 19047, 20149.

Tanh_tuned: ошибок нет.

Номера соответствуют строкам исходного CSV с учётом строки заголовка.

Полный список объектов, меток и предсказаний находится в predictions.csv.

Приложение В. Порядок воспроизведения

1. Запустить `python lab4_recalculation.py --output-dir lab4_reworked`.
2. Открыть `lab4_reworked/water_variant17_lab4_work.xlsx` в Excel, выполнить полный пересчёт и сохранить книгу.
3. Запустить `python verify_lab4.py` для построчной проверки.
4. Запустить `python build_lab4_report.py` для графиков и отчёта.